NETBOX PLUGIN

User PIN – guide

A complete guide for NetBox 4.7+, checked against the code

Version 2.0.1 · 2026-10-31
Installation · key · e-mails · PIN and 2FA · users · roles · approval · settings · areas · protection and exceptions · PIN-only people · working time · commands · troubleshooting · changes
github.com/ArK761/NetBox_UserPIN

Contents

1. What the plugin needs and what it adds to NetBox
    - 1.1 Requirements
    - 1.2 Dependencies (Python packages)
    - 1.3 What the plugin adds to NetBox
2. Installation
    - 2.1 1. Packages
    - 2.2 2. The encryption key
    - 2.3 3. Configuration
    - 2.4 4. Migrations and restart
    - 2.5 5. Check that everything runs
    - 2.6 6. First steps in the interface
    - 2.7 What happens when the configuration is wrong
    - 2.8 Optional settings in the configuration
3. Updating the plugin and NetBox
    - 3.1 Updating just the plugin (one line)
    - 3.2 Upgrading NetBox itself
4. The encryption key
    - 4.1 What the key protects
    - 4.2 How to generate a key
    - 4.3 Which key the plugin accepts
    - 4.4 Writing it into the configuration
    - 4.5 Backing up the key
    - 4.6 Replacing the key (rotation)
    - 4.7 What `userpin_check` prints
5. E-mails: how the plugin sends mail
    - 5.1 When an e-mail is really sent
    - 5.2 What the plugin sends from
    - 5.3 What an e-mail looks like
6. The Mail page
    - 6.1 E-mail settings (fields)
    - 6.2 Notifications and forgotten PIN
    - 6.3 Status and the connection test
    - 6.4 The test e-mail
7. Allowed e-mail domains
    - 7.1 Adding and verifying a domain
8. Which e-mails the plugin sends (the PIN core)
9. E-mails: when they do not arrive
10. My PIN
    - 10.1 Who may use a PIN
    - 10.2 The My PIN page
    - 10.3 Setting and changing the PIN
    - 10.4 Unlocking
    - 10.5 Wrong attempts and lockout
    - 10.6 Validity and rotation
    - 10.7 Forgotten PIN (self-service)
11. Two-factor authentication (2FA)
    - 11.1 Backup codes
    - 11.2 Lost phone
    - 11.3 Confirming administrative actions
12. Users (the PIN page)
    - 12.1 Filters
    - 12.2 Columns
    - 12.3 Actions for a user
    - 12.4 Who may manage whom
    - 12.5 Emergency 2FA reset
13. Roles: CORE and departments
    - 13.1 Who sees and does what
    - 13.2 An invitation to a role
    - 13.3 Minimum and the end of a role
    - 13.4 CORE deputies (the page CORE (deputies), the master only)
    - 13.5 Departments
    - 13.6 Moves between departments
    - 13.7 My roles
14. Four-eyes approval
    - 14.1 Switching on and categories
    - 14.2 The course of a request
15. Audit log
    - 15.1 What is recorded
16. Settings
    - 16.1 General
    - 16.2 Access & PIN policy
    - 16.3 Rotation
    - 16.4 Unlock
    - 16.5 Protection
    - 16.6 Four-eyes approval
    - 16.7 Protected areas
    - 16.8 What is on other pages
17. Protected areas
    - 17.1 The basic idea
    - 17.2 The Protected areas page
    - 17.3 The area detail
    - 17.4 A group: rights
    - 17.5 A group: the records it sees
    - 17.6 Fields
    - 17.7 Tabs of the detail
    - 17.8 An access request
    - 17.9 My access
    - 17.10 What is enforced
    - 17.11 An area of a plugin

18. Global protection (ON / OFF) and keepers
    18.1 ON and OFF
    18.2 Protection keepers
    18.3 What the users see
    18.4 Emergency on the server
19. Exceptions
    19.1 An exception on and off (the switch)
    19.2 The end of an exception (date and time)
    19.3 Searching and a list for many users
    19.4 Adding an exception
    19.5 What a user with an exception may use
20. Section managers
21. People with a PIN only (PIN-only)
    21.1 A new access (the form)
    21.2 The person's first sign-in
    21.3 Regular sign-in: one shared address
    21.4 What the person sees and cannot do
    21.5 Wrong PINs
    21.6 Actions at a person
    21.7 The person's session
    21.8 E-mails both ways
22. Working time
    22.1 The rules
    22.2 What happens outside the time
23. Server commands
24. Check: pin_check.sh
25. Security model
26. Using it from another plugin
    26.1 An area of a plugin
27. Troubleshooting
28. Changes
    28.1 2.0.1: permissions of exceptions
    28.2 Keys and security
    28.3 The new model of protection
    28.4 Section managers and requests
    28.5 PIN-only people
    28.6 Working time
    28.7 Tools
    28.8 What the plugin does not do yet

1. What the plugin needs and what it adds to NetBox

Who: the server administrator (command line and `configuration.py` access).
When: before the first installation; at every plugin or NetBox update.
Depends on: NetBox 4.7.0 or newer, Python 3.12+, a running `netbox-rq` worker, internet access (GitHub, PyPI) during installation.

1.1 Requirements

What	Details
NetBox	version **4.7.0 or newer** (the plugin has `min_version = 4.7.0` and no upper limit). With an older NetBox the plugin does not load.
Python	3.12 or newer (`requires-python >= 3.12`).
`netbox-rq` worker	The plugin registers a system job **User PIN role housekeeping** that runs **every 5 minutes**. It expires invitations, moves and exceptions, sends reminders, returns rights after a temporary hand-over, announces the end of PIN-only accesses and sends the summaries of changes. The e-mails about the changes of a PIN-only person also go through the `default` queue. When the worker does not run, these things are not done.
Database	the tables of the plugin are created by `migrate` (migrations 0001 to 0043).

1.2 Dependencies (Python packages)

Package	What it is for	When missing
`argon2-cffi >= 23.1`	PIN hashing (Argon2id)	NetBox does not start: `ModuleNotFoundError: No module named 'argon2'`
`cryptography >= 42`	AES-256-GCM encryption and key derivation (HKDF)	NetBox does not start
`segno >= 1.5`	the QR code of the 2FA setup	the pages of the plugin do not load (import error of `segno`)
`holidays >= 0.50`	public holidays for the working time of the PIN-only people	the plugin runs, only days and hours are checked (no holidays)

Mind it: installing with `--no-deps` does not install the dependencies. On a server where the plugin has never been, install them separately (see the Installation chapter).

1.3 What the plugin adds to NetBox

- **The menu "User PIN"** with the items: *My PIN*, *My roles*, *My access*, *Test unlock*, *People with a PIN only*, *Users*, *Departments*, *Protected areas*, *Approvals (four eyes)*, *Audit log*, *Settings*, *Mail*, *CORE (deputies)*. Which item a user sees depends on the role (see the Roles chapter).
- **The address of the plugin**: `/plugins/user-pin/`.

- **Two middleware classes** that are added automatically: `PluginLanguageMiddleware` (language of the plugin pages) and `AreaGuardMiddleware` (the guard of the protected areas and of the global protection). They are not written into `configuration.py`.
- **Three event types** in the NetBox notifications (the bell): *PIN four-eyes approval requested*, *PIN area access requested*, *PIN role / department change*.
- **NetBox permissions** named `User PIN: …` (deputies, settings editors, departments). The plugin creates and manages them itself: do not edit them by hand.

2. Installation

Who: the server administrator (`root` or `sudo`).
When: once, on every NetBox server separately (test, production).
Depends on: the chapter What the plugin needs; a generated key (next chapter).

2.1 1. Packages

```
source /opt/netbox/venv/bin/activate
pip install "argon2-cffi>=23.1" "cryptography>=42" "segno>=1.5" "holidays>=0.50"
pip install --upgrade --force-reinstall --no-deps "git+https://github.com/ArK761/NetBox_UserPIN.git
@2.0.0"
```

Instead of `@2.0.0` you can use any tag or commit. After the installation pip may print a warning about `python-dateutil` (see Troubleshooting): it is harmless.

2.2 2. The encryption key

Generate it and store it aside **before** you switch the plugin on. The procedure is in the chapter *Encryption key*.

2.3 3. Configuration

In `/opt/netbox/netbox/netbox/configuration.py` add `'netbox_user_pin'` to `PLUGINS` and a block to `PLUGINS_CONFIG`:

```
PLUGINS = [
    # ... your other plugins ...
    'netbox_user_pin',
]

PLUGINS_CONFIG = {
    # ... the settings of the other plugins ...
    'netbox_user_pin': {
        'encryption_keys': {'k1': '<your key>'},
        'active_key_id': 'k1',
    },
}
```

Mind the two blocks. If you already have other plugins in `PLUGINS` and `PLUGINS_CONFIG`, the line and the block are **added**, not replaced. Check the commas and apostrophes as well: an error in the file stops the whole NetBox.

2.4 4. Migrations and restart

```
cd /opt/netbox/netbox
python manage.py migrate netbox_user_pin
sudo systemctl restart netbox netbox-rq
```

`migrate netbox_user_pin` creates the tables of the plugin. If it reports *No installed app with label 'netbox_user_pin'*, the plugin is not in the `PLUGINS` list.

2.5 5. Check that everything runs

```
python manage.py showmigrations netbox_user_pin | tail -3      # every line has [X]
python manage.py userpin_check                                  # after a fresh install: OK: 0 ...
failed: 0
systemctl is-active netbox netbox-rq                            # active, active
```

User PIN appears in the NetBox menu. In *Settings* there is the card *Encryption key* with the active key and its fingerprint.

2.6 6. First steps in the interface

1. Sign in as superuser → **User PIN → My PIN** → *Set PIN* and *Set up 2FA* (administrative actions need 2FA).
2. On the **Mail** page set up sending and verify at least one domain (see the E-mails chapter).
3. On the **Users** page allow those who are to use a PIN.

2.7 What happens when the configuration is wrong

At start the plugin checks the keys and **NetBox does not start** if there is a problem (a key is never generated or switched silently). So always watch `journalctl -u netbox -n 50` after editing `configuration.py`.

Start-up error	Cause
`encryption_keys must be a non-empty dict`	`encryption_keys` is missing or empty
`invalid key id '…'`	the key id is empty or contains a colon (`:`)
`must be a base64 32 byte key or a secret of at least 32 characters`	the key is shorter than 32 characters and is not a valid base64 key
`several encryption keys are configured; set 'active_key_id'`	there are several keys and `active_key_id` is missing
`active_key_id '…' is not in encryption_keys`	the active id is not among the keys

2.8 Optional settings in the configuration

Setting	Default	Meaning
`encryption_keys`	– (required)	a dictionary `{key_id: key}`
`active_key_id`	the only key	the key for new encryptions; required when there are several keys
`argon2_time_cost`	3	Argon2id iterations

`argon2_memory_cost`	65536	Argon2id memory in KiB (64 MiB)
`argon2_parallelism`	4	Argon2id lanes
`cli_venv`	/opt/netbox/venv	the path the plugin shows in the emergency 2FA reset command
`cli_netbox_dir`	/opt/netbox/netbox	the same for the NetBox directory

Everything else is set in the interface (**User PIN → Settings** and **Mail**).

3. Updating the plugin and NetBox

Who: the server administrator.
When: at a new version of the plugin or NetBox.
Depends on: a database backup (before a migration), the pinned version in `local_requirements.txt`.

3.1 Updating just the plugin (one line)

```
sudo bash -c 'source /opt/netbox/venv/bin/activate && pip install --upgrade holidays && pip install --upgrade --force-reinstall --no-deps "git+https://github.com/ArK761/NetBox_UserPIN.git@<tag-or-commit>" && cd /opt/netbox/netbox && python manage.py migrate netbox_user_pin && systemctl restart netbox netbox-rq'
```

Production: back up the database before the migration, for example `sudo -u postgres pg_dump netbox | gzip > /root/netbox-$(date +%F-%H%M).sql.gz`. Do not touch the keys in `configuration.py` meanwhile.

3.2 Upgrading NetBox itself

`upgrade.sh` creates the virtual environment **anew** and installs only what is in `requirements.txt` and `local_requirements.txt`. A plugin that is not listed there disappears after the upgrade. Lines in `/opt/netbox/local_requirements.txt`:

```
netbox-librenms-plugin
netbox-qrcode
git+https://github.com/ArK761/NetBox_UserPIN.git@2.0.0
```

- A package from PyPI needs only its name. A plugin from GitHub needs the whole link.
- A line without `@…` installs the **latest commit of the `main` branch** (not the latest release; pip does not know GitHub Releases, only branches, tags and commits). If the code of a plugin is on another branch, keep the pinned commit.
- After the upgrade check the plugin versions (the badge *User PIN vX · commit*) and run `userpin_check`.

4. The encryption key

Who: exclusively the server administrator (the key is only in `configuration.py`). Nobody in the interface sees it; *Settings* shows only its name and fingerprint.
When: you generate it before the first installation. You replace it on a suspected leak or by the company policy.
Depends on: everything the plugin encrypts cannot be read without the key. So the key must be backed up separately from the database.

4.1 What the key protects

Under the key are stored (AES-256-GCM encryption, bound to the particular user):

- the PIN hash of every user (the PIN is first hashed with Argon2id and a salt and only the hash is encrypted),
- the 2FA (TOTP) secret of every user,
- the 2FA backup codes,
- the SMTP password from the Mail page.

The key is also used to compute digests (these are not encrypted): the lookup index of the PIN of the PIN-only people and the verification codes of the domains. A database dump alone therefore contains nothing usable, not even something to brute-force offline.

4.2 How to generate a key

```
# while the plugin does not run yet – plain Python:
python3 -c "import base64,os; print(base64.urlsafe_b64encode(os.urandom(32)).decode())"

# or any long random text:
openssl rand -base64 64 | tr -d '\n'

# once the plugin runs:
cd /opt/netbox/netbox && python manage.py userpin_generate_key --key-id k1
```

`userpin_generate_key` only prints the key, it stores it nowhere. `--key-id` is only the name it gets in the configuration (`k1` by default).

4.3 Which key the plugin accepts

Form of the key	What happens
a base64 (urlsafe) string that is exactly 32 bytes	used directly
any other string with **at least 32 characters**	a 32-byte key is derived from it (HKDF-SHA256)
a shorter string	NetBox does not start (an error at start)

A longer key is not stronger: the encryption always uses 256 bits. What matters is that it is **random** (not an invented password). The key name (`k1`) must not be empty or contain a colon.

4.4 Writing it into the configuration

```
PLUGINS_CONFIG = {
    'netbox_user_pin': {
        'encryption_keys': {'k1': '<key>'},
        'active_key_id': 'k1',      # may be left out with a single key
    },
}
```

4.5 Backing up the key

> - Store the key **offline** (a password manager, a safe) and **never in the same backup as the database**. Whoever has both can decrypt.
> - **When the key is lost**, the NetBox data stays intact, but everything the plugin stored under the key is irretrievably gone: PINs, 2FA, backup codes, the SMTP password and the sign-in of the PIN-only people. Everybody has to set them up again.
> - Check that the backup matches: in *User PIN → Settings* there is the card **Encryption key** with *Active key*, *Fingerprint* and *Configured keys*. Compare the fingerprint with the note at the offline backup.

4.6 Replacing the key (rotation)

During a rotation every encrypted value is re-encrypted with the new key. The old key stays in the configuration until everything has moved.

1. **Back up the database** (to be safe).
2. Generate a new key and store it aside.
3. In `configuration.py` add the new key **next to the old one** and make it active:

   ```
   'encryption_keys': {'k1': '<old>', 'k2': '<new>'},
   'active_key_id': 'k2',
   ```

4. Restart NetBox: `sudo systemctl restart netbox netbox-rq`.
5. Re-encrypt: `python manage.py userpin_rotate_key`. It runs in one transaction: if even one value cannot be decrypted, **nothing changes** and the command prints what failed. On success it writes a key-rotation event to the audit log.
6. Verify: `python manage.py userpin_check`.
7. **Remove the old key only when `userpin_check` prints** *Everything decrypts with the active key; old keys can be removed.*

Why not delete the old key at once: the index of the PIN-only people moves to the new key only at each person's next sign-in. Until a person signs in, they wait for the old key. The command prints it (*PIN-only accesses still on an old key: N*) and then *Do NOT remove the old key yet*.

4.7 What `userpin_check` prints

Line	Meaning
`Active key: k2 fingerprint …`	the active key and its fingerprint

`OK: N, encrypted with a non-active key: M, failed: F, PIN-only accesses still on an old key: G`	N values decrypted; M still use an old key; F cannot be decrypted; G PIN-only people wait for a sign-in
an error (the command ends with an error)	at least one value cannot be decrypted: a wrong or missing key. Do not change `k1` and send the output

> The plugin never creates or switches a key by itself. A missing or wrong key ends with an error, not with a silent change.

5. E-mails: how the plugin sends mail

> **Who sets it up:** the master and CORE deputies with the permission to edit settings (the *Mail* page). The NetBox server (`EMAIL` in `configuration.py`) is set up by the server administrator.
> **When:** right after the installation. Without mail set up the plugin **sends no e-mails**: recovery of a forgotten PIN, invitations to roles and notifications do not work.
> **Depends on:** a sender + an SMTP server, a **verified domain** of the recipient, an address in the recipient's account.

5.1 When an e-mail is really sent

An e-mail to a user goes out only when **all four** conditions are met:

1. mail is set up (see below),
2. the user has an e-mail address in the account,
3. the domain of that address is among the **verified** domains (the Domains chapter),
4. for security notifications: the switch *Security notifications by e-mail* is on.

If any of them is missing the e-mail is **silently not sent** and the action is carried out. The e-mail state of a user is seen on the *Users* page in the E-mail column: *no e-mail* (the address is missing), *domain* (the address is outside the verified domains). Otherwise the address itself is shown. When sending fails (e.g. the server is unreachable) an *E-mail failed* event with the error is written to the audit log; the action that triggered the e-mail is not spoiled.

5.2 What the plugin sends from

Source	When it is used	What must be set
SMTP from this page	when **both the SMTP server and the sender address** are filled in	the Mail page → E-mail settings
NetBox configuration	when the SMTP server on the Mail page is **not** filled in and NetBox has an `EMAIL` block in `configuration.py`	`EMAIL['SERVER']` and `EMAIL['FROM_EMAIL']` (or the sender address from the Mail page)
nothing	neither is set	the state *not configured*: the plugin sends no e-mails

Users never enter any mail credentials: one service account is used.

5.3 What an e-mail looks like

- Plain text (no HTML). The subject has the prefix `[NetBox PIN]` (e.g. `[NetBox PIN] Your PIN was changed`).
- At the end there is the footer *NetBox User PIN. This is an automatic message.*
- The text is in the language from *Settings → Language* (for PIN-only people in the language set for their access).
- Sender: *Sender name* + the address (the default name is *NetBox PIN*).

6. The Mail page

Who sees it: CORE (master and deputies). Nobody else has the page. Exception: when there are protection keepers and active CORE deputies, a superuser who is not a CORE deputy does not see it.

Who can change it: the master and a CORE deputy with the permission *Can edit settings* (switched on by the master on the CORE (deputies) page). Others see the page marked *Read-only view*.

Confirmation: saving needs the PIN (and 2FA when *Require 2FA also for settings* is on). The connection test and the test e-mail need no confirmation.

Four eyes: when approval is on and the option "... for settings and mail settings" (on by default) is on, saving the mail settings and adding a domain create a request for a second person.

6.1 E-mail settings (fields)

Field	Default	Meaning
Sender name	NetBox PIN	shown as the sender's name
Sender e-mail address	empty	e.g. `netbox@company.com`. Required when an SMTP server is filled in
SMTP server	empty	e.g. `mail.company.com`. Empty = the NetBox configuration is used
SMTP port	25	1 to 65535
SMTP timeout (s)	10	1 to 120
Encryption	Off	*Off (usually port 25), STARTTLS (usually port 587), SSL/TLS (usually port 465)*
Automatic TLS	on	with encryption off, STARTTLS is used when the server offers it
SMTP authentication	off	when on, an SMTP user and a password are needed
SMTP user	empty	the service account the mail is sent from
SMTP password	empty	stored **encrypted with the key** and never shown; an empty field on editing = keep the current one. Required with authentication on if none is stored yet

On saving, the plugin checks: an SMTP server without a sender address is an error (*A sender address is needed together with the SMTP server*), authentication without a user or a password is an error.

6.2 Notifications and forgotten PIN

Field	Default	Meaning
Security notifications by e-mail	on	PIN set / changed / reset, lockout, upcoming expiry, 2FA changes, backup codes viewed. Switching it off does not affect the other mails (codes, invitations, roles, areas)
Self-service recovery of a forgotten PIN	on	an e-mail code **and** a 2FA code together
Recovery code validity (minutes)	15	5 to 60; applies also to the domain verification code and to the code for showing backup codes

6.3 Status and the connection test

The **Status** card shows *Used*: *SMTP from this page*, *NetBox configuration* or *not configured*, and accordingly the server, the account (or *no authentication*) and the sender. The **Test connection** button connects to the SMTP server from the page (and logs in) and writes the result. It works only for the SMTP from the Mail page, not for the NetBox configuration.

6.4 The test e-mail

Choose a recipient and press **Send test e-mail**. Only users who **may use a PIN and have an e-mail in a verified domain** can be chosen; the picker shows the account, the name and the address. If nobody qualifies, the page says so. Result: *Test e-mail sent to …* or an error pointing to the audit log.

7. Allowed e-mail domains

Who: whoever may change the mail settings (see above).

When: once for each domain that is to receive e-mails. Add a new one also when a company with another domain joins.

Depends on: mail must be set up and somebody with an address in that domain must be able to read the code.

Mail goes **only to addresses in verified domains**. Until a domain is verified, e-mails to it are not sent (the list says *not verified – no e-mails yet*; with no domains it says *No domain – no e-mails are sent*).

- A domain is written e.g. `@company.com` (the at sign may also be left out; the plugin strips it and converts to lower case). It must contain a dot and no spaces.
- **The match is exact:** `company.com` lets `jan@company.com` through, but **not** sub-domains (`jan@mail.company.com`) or look-alikes (`company.com.evil.com`).

7.1 Adding and verifying a domain

1. In the card *Allowed e-mail domains* write the domain and press **Add**. The dialog *Verify domain* opens at once. (With four-eyes approval on, the addition must be approved first.)
2. **1. Address in this domain:** write one address that ends with this domain and press **Send code**. An e-mail with an **8-digit code** is sent.
3. **2. Code from the e-mail:** enter the code and press **Verify**.

The code is valid for **Recovery code validity** minutes (15 by default) and can be entered at most **5 times** (then a new one must be sent). If you already have a code, the dialog has the link *Already have a code? Enter it directly*. On success the plugin stores who verified the domain and through which address. Errors: *The address must end with @domain*, *No mail server is configured*, *The code is wrong or expired*.

- **Verify again** is available at any time (e.g. for a new address); it does not cancel the domain.
- **Removing** (the button at the domain, with a confirmation) cancels the domain at once: e-mails to it stop. Removing does not need four-eyes approval.
- Adding, verifying and removing are written to the audit log.

8. Which e-mails the plugin sends (the PIN core)

These are the mails about the PIN and 2FA itself. The e-mails about roles, approvals, areas and PIN-only accesses are described in their own chapters.

Subject (in English; translated into the set language)	Who gets it and when	Does the "Security notifications" switch turn it off?
Your PIN was set / was changed / was reset / was recovered	the user, after setting, changing, a reset by an administrator, recovery of a forgotten PIN	yes
Your PIN was suspended / Your PIN is active again	the user, when an administrator suspends or restores the PIN	yes
Your PIN is locked	the user, after exceeding the number of wrong attempts	yes
Your PIN expires soon	the user, **once** when entering the warning period (14 days before the expiry by default)	yes
Two-factor authentication enabled / disabled / reset	the user, on a 2FA change	yes
Your backup codes were viewed	the user, after viewing the backup codes	yes
Reset of your PIN / two-factor authentication started	the user, when an administrator starts a reset (the user must confirm it in NetBox)	yes
Verification code	an 8-digit code to show the backup codes without the phone (validity by the setting, at most 5 attempts, a new code after 60 s at the earliest)	no
PIN recovery code	an 8-digit code to recover a forgotten PIN	no
Domain verification	the code to verify a domain (to the entered address)	no
Test e-mail	the chosen recipient from the Mail page	no

> The switch *Security notifications by e-mail* affects only the first seven rows. Codes, invitations and the mails about roles, approvals, areas and PIN-only accesses always go when the conditions at the beginning of this part are met (mail, address, verified domain).

9. E-mails: when they do not arrive

Symptom	What to check
Nobody gets anything	The Mail page → card *Status*: is *Used* something other than *not configured*? Then **Test connection** and a **test e-mail**.
The connection test reports an error	server, port, encryption (port 25 with no encryption, 587 STARTTLS, 465 SSL/TLS), credentials; the SMTP server must be reachable from the NetBox server.
The test passes, somebody gets no mail	In the *Users* list see the E-mail column: *no e-mail* or *domain* means the address is missing or its domain is not verified.
I added a domain but no mail arrives	The domain must be **verified**: the list must say *verified*, not *not verified*.
The code does not arrive when adding a domain	Mail is not set up (*No mail server is configured*) or the address is wrong. The audit log has *E-mail failed* with the error.
Security notifications do not arrive, the others do	The switch *Security notifications by e-mail* is off.
The summaries of changes of PIN-only people do not arrive	`netbox-rq` is not running (the queue sends them).

All sent and failed e-mails are in the *Audit log* (the events *E-mail sent* and *E-mail failed*, without the content of the e-mail).

10. My PIN

Who: every user who may use a PIN. Every signed-in user sees the page *My PIN*.
When: at first use, when the validity expires (after 180 days by default), after a reset.
Depends on: the permission to use a PIN (below); 2FA and an e-mail in a verified domain for recovery of a forgotten PIN.

10.1 Who may use a PIN

It is decided by the setting *Who may use a PIN* (Settings → Access & PIN policy) and by the flag of each user (the *Users* page):

Flag of the user	Mode "only explicitly allowed" (default)	Mode "all except denied"
Allowed	may	may
Denied	may not	may not
Default (new accounts)	may not	may

- A **superuser** always may.
- Whoever holds a **role** in the PIN administration (CORE, head, delegate) or awaits the acceptance of an invitation always may: they need the PIN to accept and to work.
- Whoever may not sees on the page *My PIN*: *Permitted to use a PIN: No – ask an administrator*.

10.2 The My PIN page

Item	What it shows
Permitted to use a PIN	Yes / No – ask an administrator
PIN	Active / Not set / Change required
Last changed; Valid until	a date; the days left (*rotation every N days*) or *expired*
Last successful use	date and time
Lockout	until when the account is locked (after exceeding wrong attempts)
Unlocked now	the scope of the unlock (or *everything*)
Required length	N digits

Buttons: **Lock now** (locks everything in this session at once), **Test rotation** (forces a change of your own PIN so you can try the procedure), **Forgot PIN**, **Change PIN** or **Set PIN**. Below is *Recent activity* (time, event, scope).

Notices on the page: *Your PIN expires in N days* (with a one-click change), *Two-factor authentication is not set up*, *Your PIN is suspended*, *an administrator started a reset*, *an invitation to a role*.

10.3 Setting and changing the PIN

- **Digits** only, exactly the length in the settings (*PIN length*, 6 by default, range 4 to 12). An old PIN of another length stays valid until changed.
- **Weak PINs** are rejected (when *Block weak PINs* is on, the default): repeated digits and patterns (`111111`, `121212`, `123123`), runs up or down (`123456`, `654321`) and the own list *Additional blocked PINs* (one per line or comma separated).
- Nobody, not even an administrator, can read the PIN: an administrator can only reset it.
- A PIN change or reset **invalidates every existing unlock** of the user (in all sessions).

10.4 Unlocking

A protected page asks for the PIN (*Enter your PIN*). After the correct PIN it is unlocked for *Unlock duration* minutes (15 by default; 1 to 1440). With *Extend unlock on activity* on (the default) every view of a protected page restarts the timer, so the unlock expires after inactivity. *Unlock scope*: **one unlock for everything** (default) or **a separate one per scope** (plugin / area). *Require 2FA on every unlock* (off by default) also requires a 2FA code; a user without 2FA is then not unlocked.

The unlock does not apply when the PIN is suspended, has expired, or the user may no longer use a PIN. The *Test unlock* page in the menu is protected by its own scope *test* and serves to try it out.

10.5 Wrong attempts and lockout

After *Max failed attempts* (5 by default; 1 to 50) the account is locked for *Lockout duration* minutes (30 by default; up to 7 days). Wrong PINs and wrong 2FA codes are counted **separately**: a correct PIN does not reset wrong 2FA codes and vice versa. Counting happens under a row lock, so parallel guessing cannot bypass the lockout. A locked user sees *Too many failed attempts. Try again after …*. An administrator (*Clear lockout*) or the passing of time unlocks.

10.6 Validity and rotation

A PIN is valid for *PIN max age* days (180 by default; 1 to 3650). *Warn before expiry* (14 days by default) sends an e-mail **once** and shows a notice. After the expiry, or when an administrator forces a change, the PIN must be changed before the next use (*Change required*). Periodic changes are a company policy; NIST SP 800-63B-4 does not recommend them: set it by your rules.

10.7 Forgotten PIN (self-service)

1. *My PIN → Forgot PIN* (or the link *Forgot your PIN?* on the unlock page) → **Send code**. It is sent only to an address in a verified domain. An 8-digit code is sent; valid for *Recovery code validity* minutes, at most 5 attempts, a new code after 60 s at the earliest.
2. Enter the e-mail code **and** a 2FA code (or a backup code) and the new PIN. On success the e-mail *Your PIN was recovered* arrives.

Recovery is not offered (the page lists the reasons) if: it is switched off, the user may not use a PIN, NetBox cannot send e-mails, the account has no e-mail, the e-mail is not in a verified domain, or 2FA is not set up. Then an administrator resets the PIN. Recovery **never bypasses 2FA**.

11. Two-factor authentication (2FA)

Who: every user sets it up themselves (*My PIN → Set up 2FA*).

When: as soon as possible: without 2FA administrative actions cannot be confirmed, a forgotten PIN cannot be recovered, and a role (head, delegate, CORE deputy) cannot be accepted.

Depends on: an authenticator app on the phone with TOTP (Microsoft / Google Authenticator etc.), the package `segno` (QR code).

1. *Set up 2FA* → **1. Scan with your authenticator app** the QR code (or enter the key manually: *Can't scan? Enter this key manually*).
2. **2. Enter the 6-digit code** from the app. The backup codes are shown at once (*Save these codes now*, buttons *Copy* and *I have saved them*).

- A TOTP code is accepted **only once** (replay protection).
- **Disabling 2FA** (*My PIN → Disable 2FA*) needs the PIN and a 2FA code. Without 2FA administrative actions cannot be confirmed and a PIN cannot be recovered by yourself. The e-mail *Two-factor authentication disabled* arrives.

11.1 Backup codes

One-time codes instead of the app (e.g. after losing the phone). *My PIN → Show backup codes* needs the PIN and either a **2FA code** or (without the phone) a **verification code from an e-mail** (*No phone at hand? Get a verification code by e-mail*). The codes are shown only in the browser (used ones are marked *used*); **Generate new codes** invalidates all the old ones. The codes themselves are never e-mailed. After every viewing the user gets an e-mail and it is written to the audit log.

11.2 Lost phone

1. Use a backup code.
2. If you have none either: another administrator starts **Reset 2FA** (the user confirms it with the PIN and sets up the new phone).

3. If nobody can do it (e.g. the master without the phone and codes): the server administrator runs `python manage.py userpin_reset_2fa <user>`. The 2FA is removed at once and it is written to the audit log. The command with the exact paths is shown by *My PIN* and *Users → Emergency: server command*.

11.3 Confirming administrative actions

An administrative action is confirmed in a dialog directly on the page (*Confirm your identity*): *Your PIN* and a *2FA code* (6 digits from the app, or a backup code). The confirmation is valid for *Administrative window* minutes (5 by default; 1 to 60) and the action continues without leaving the page. Actions are of two levels:

Level	What it needs	Examples
PIN only (default)	the PIN; with *Require 2FA also for settings* on, 2FA as well	saving Settings and Mail, allow/deny a PIN, clear a lockout, force a change, moves between departments
PIN + 2FA	always 2FA as well	resets, suspension, roles, dissolving a department

If you switch off *Require PIN + 2FA for administrative actions*, the PIN alone is enough. The administration of protected areas is confirmed differently: the PIN is unlocked once and 2FA is asked only every *2FA asked again after* hours (8 by default; 1 to 168), not at every change.

12. Users (the PIN page)

Who sees it: CORE (master and deputies) everybody; heads and delegates only their department. An ordinary user does not have the page.
When: in PIN administration: allowing, resets, suspension, clearing lockouts.
Depends on: the administrator's role (the Roles chapter); a PIN or PIN + 2FA confirmation before the first action.

Menu **User PIN → Users**, heading *User PINs*.

12.1 Filters

- **Search** (name, e-mail).
- **Status:** All users / PIN active / No PIN yet / No 2FA / E-mail issue (missing / domain) / Locked out / Suspended / Not permitted.
- **Department:** all / without a department / a given one. **Role:** all / master / head / delegate / invited / without a role.
- **View:** *List* or *By departments* (a tree: head → delegates → members; department colours: green = a head and at least 2 delegates, yellow = delegates missing, red blinking = no head).

12.2 Columns

Column	Values
User	the account (+ the name when *Show first and last name* is on); tags *you*, *master*, *head*, *delegate*, *invited*, *inactive*
Permitted	*allowed* / *denied* / default (decided by the mode in Settings); *Yes* / *No* = whether the user may use a PIN now

PIN	*Active*, *Not set*, *must change*, *suspended*, *PIN reset pending*, *2FA reset pending*
2FA	On / Off
E-mail	the address, or *no e-mail* / *domain* (outside the verified domains): without it e-mails do not arrive
Valid until	a date or *expired* (and the rotation every N days)
Last used; Lockout	time; until when it is locked

12.3 Actions for a user

Action	Confirmation	What it does
Allow / **Deny** / **Default**	PIN	sets the flag of the user (see the table in the My PIN chapter). With four-eyes approval on for the category "allowing PIN use" it creates a request
Reset PIN	PIN + 2FA	starts a reset; the user confirms it in their own session with a **2FA code** and sets a new PIN. Valid for *Administrator reset valid* hours (24 by default). The old PIN works until the user confirms
Reset 2FA	PIN + 2FA	starts a reset; the user confirms it with the **PIN** and sets up 2FA on a new phone
Cancel reset	PIN + 2FA	cancels a pending reset
Suspend / **Lift suspension**	PIN + 2FA	blocks the PIN at once (suspected leak, absence), alone, without a second person; the user gets an e-mail
Clear lockout	PIN	clears the lockout after wrong attempts
Force change	PIN	the user must change the PIN at the next use
Remove PIN	PIN + 2FA	only for a user **without 2FA** who cannot confirm a reset: the PIN is removed and the user sets a new one at the next prompt. With 2FA a reset must be started

> A reset never reveals the new PIN to the administrator. The administrator starts the reset, the user finishes it.

12.4 Who may manage whom

Administrator	Whom they may manage
Master (superuser)	everybody except themselves
CORE deputy	everybody except the master, other CORE deputies and themselves
Department head	the members of their department except themselves and other heads
Department delegate	the members of their department who have **no role**

Nobody manages themselves (you change your own PIN in *My PIN*). On a forbidden action the page answers: *Delegates cannot manage the master, other delegates or themselves.*

12.5 Emergency 2FA reset

In the *Actions* menu at a user there is *Emergency: server command (2FA)*: a ready-made `userpin_reset_2fa` command to run as root. Use it only when the normal way (a reset confirmed by the user with the PIN) is not possible. The 2FA is removed at once and the event is written to the audit log.

13. Roles: CORE and departments

Who: the master (superuser) appoints CORE deputies. CORE creates departments and appoints heads. Heads invite delegates. A delegate manages the members without a role.

When: when PINs are managed not by one person but by several; after the installation the master alone is enough.

Depends on: a PIN and 2FA of every role holder, an e-mail in a verified domain (the invitation), mail set up, a running `netbox-rq` (reminders, expiry of invitations).

```
CORE        master (superusers) + CORE deputies        manages everybody
Department  head + delegates (min. 2) + members          the PINs of the members of the department
```

13.1 Who sees and does what

Menu	Master	CORE deputy	Head	Delegate	User
My PIN, My roles	✓	✓	✓	✓	✓
Users	all	all	own department	own department	–
Departments	all	all	own: invite delegates, hand over, ask for a move	own: read, with a second delegate invite and hand over	–
Audit log, Approvals	all	all	own department	own department	–
Settings, Mail	edit	read (edit with the permission)	–	–	–
CORE (deputies)	✓	–	–	–	–

Who may manage whom (the actions on the Users page) is in the Users chapter.

13.2 An invitation to a role

Every role (CORE deputy, head, delegate, temporary head, replacement) is an **invitation**:

1. The inviter chooses a user and the **validity of the invitation: 12, 24, 36 or 48 hours** (the page rejects any other value).
2. The invitee gets an e-mail explaining the role and the responsibility, with a link and a code, and a notice in the bell.
3. The invitee opens the invitation. If they have no PIN or 2FA yet, the page first sends them to **set them up** (*First set up* PIN and 2FA) and then they return.
4. The invitee **accepts** the invitation with the code from the e-mail, their PIN and a 2FA code (the code may be entered at most 5 times), or **declines** it. Only by accepting does the role start.

Who can be invited: an active user who is not a superuser (the master needs no role), has no denied PIN, **has an e-mail in a verified domain and mail is set up** (otherwise the invitation would not arrive; the page refuses and says why), has no same role or open invitation. A CORE deputy cannot hold

a department role; a person in another department must be moved first. **A delegate can be invited only when the department has a head**: first the head, then the delegates. If an invitation expires it can be sent again (*Send again*) or cancelled.

13.3 Minimum and the end of a role

- **Minimum:** with four-eyes approval on at least **2 CORE deputies**; every department **a head and at least 2 delegates**.
- **Resignation:** the role holder gives the role up in *My roles*. Below the minimum the role stays in force (*resigning*) until a replacement accepts. The superiors are told by e-mail (the head for a delegate, CORE for a head, the master for a CORE deputy).
- **Removal / replacement** (the button *Remove / replace*): below the minimum only together with a replacement, the old person stays until the replacement accepts. A **regular head** ends at once (CORE manages the department meanwhile, until a new head accepts the invitation).
- A department without an active head reports *The department has no active head – CORE manages it until a head accepts the invitation.*

13.4 CORE deputies (the page CORE (deputies), the master only)

- **Invite a deputy** (with the option *Can edit settings*).
- At a deputy: **Allow settings** / **Make read-only** (switches whether they may change Settings and Mail), **Remove / replace**, **Send again**, **Cancel**. Below is *History*.
- The page shows the number of active deputies and whether four-eyes approval is on (it needs at least 2).
- A deputy sees all users and departments (after a PIN + 2FA confirmation), creates departments and appoints heads. They do not manage the master or other deputies.

13.5 Departments

The **Departments** page: a table (department, head, delegates, members; *no head – managed by CORE*), moves waiting for a decision, the number of active users without a department (only CORE manages them) and the form **New department** (the name must be unique). The department detail:

Part	What it does and who
Head and delegates	a table (person, role, status; *temporary head* with "instead of ..."); *Appoint the head* / *Replace the head* (CORE); *Invite a delegate* (head, CORE; a delegate needs a second delegate, the head or CORE to confirm)
Temporary hand-over of the head	an invitation of a member of the department to be a **temporary head until a date** (holiday, illness). The member must **accept** it. The head and CORE with no limit on length; **two delegates together at most 30 days** when the head is not available. Only one at a time. The whole department gets an e-mail and a reminder a day before. After the date the rights return by themselves (the system job every 5 minutes). Earlier: *End hand-over*
Members	a table of members (PIN, 2FA, e-mail, role); the link *Manage PINs of the members*
Moves	see below
Dissolve	CORE (PIN + 2FA). All roles end, the members get an e-mail; their own PIN is not affected
History	the changes of roles in the department

13.6 Moves between departments

- **CORE moves directly.** A head **asks** for a move (*Ask for the move*); the head of the other department decides (CORE for people without a department; if the other department has no head, CORE decides).
- A request is valid for *Approval request valid* minutes (24 hours by default); the decider **approves** or **rejects** it, the requester can **cancel** it. The decider gets an e-mail and a notice. Only one move can wait per person.
- The moved person, both heads and both departments are told by e-mail. A delegate needed for the minimum cannot be moved before a replacement arrives.
- A member can also be assigned from the form *Get somebody into this department*.

13.7 My roles

Everybody sees which department they belong to, their roles (what it means and the responsibility), a waiting invitation, the notice of a temporary hand-over and the state of a resignation, and may **give a role up**. The role is synchronised into NetBox permissions named `User PIN: …`: do not edit them by hand.

14. Four-eyes approval

Who switches it on: whoever can change Settings (the master, a CORE deputy with the permission).

When: when a sensitive change is to be confirmed by a second person.

Depends on: at least **two active CORE deputies** with a PIN and 2FA; without them it cannot be switched on.

14.1 Switching on and categories

Settings → *Four-eyes approval* (off by default). After switching on you choose what it applies to:

Category	Default	Which actions create a request
… for roles (CORE, department heads and delegates)	yes	inviting to a role, ending or replacing a role, changing the deputy's permission to edit settings, dissolving a department
… for settings and mail settings	yes	saving Settings, saving Mail, adding an allowed domain
… for allowing / denying PIN use	no	Allow / Deny / Default at a user

- **Switching on** needs at least 2 active CORE deputies. The deputies are told (the e-mail *Four-eyes approval switched on*).
- **Switching off** is **not possible** while a department exists. Switching off needs two people and **ends the CORE delegation**: the deputies are told by e-mail and from then on manage only their own PIN.

14.2 The course of a request

1. The requester makes the change: a request *Four-eyes request #N* with a reason appears. The requester confirms it with PIN + 2FA (*Confirm your request*).
2. Another eligible person gets a notice in the bell and an e-mail. The eligible are: the master and CORE deputies; for a department request also its head and delegates.

3. They open *Approvals* (or the link) and **Confirm** or **Reject** with their own PIN + 2FA. A person cannot vote twice.
4. The request page refreshes itself and both see the confirmations (who, time, IP). Right after the second confirmation the change is executed.

- A request is valid for *Approval request valid* minutes (1440 = 24 h by default; 5 to 10080) and then expires.
- The requester can **cancel** it. A rejection is announced by e-mail.
- **Break-glass** (off by default, *Allow break-glass for the master*): in an emergency a superuser may execute a request **alone** with a reason (*Execute alone*). All administrators and delegates get an e-mail and it is marked in the audit log.

The page **Approvals (four eyes)**: *Waiting* (change, requested by, confirmed, valid until; *Confirm* / *Open*) and *History* (change, requested by, confirmed by, status, finished). If approval is off, the page says so.

15. Audit log

Who sees it: CORE (master and deputies) everything; heads and delegates only the events of the people of their department and their own.
When: when investigating who did what and when.
Depends on: nothing: it is always written.

Menu **User PIN → Audit log** (*PIN audit log*). Records are only appended (they cannot be changed) and **never contain a PIN, hash or code**. The page shows 50 records per page and has the filters *User* (text: the account or whoever did the action) and *Event* (a choice of types). Columns: *Time*, *User*, *Event*, *By*, *Scope*, *Detail*.

15.1 What is recorded

Area	Events
PIN	set, changed, reset, verified, wrong, locked out, rejected (locked), lockout cleared, forced change, suspended, restored, locked, not allowed
2FA	enabled, disabled, reset, wrong code, backup code used, new backup codes, backup codes viewed
Confirmation and recovery	step-up (success and failure), recovery code sent/correct/wrong, verification code e-mailed, reset started / completed / cancelled
E-mail	e-mail sent / failed, domain added / verified / verification failed / removed
Roles and departments	invitation, acceptance, declining, expiry, resigning, ended, on leave, returned, deputy added/removed/changed, department created/deleted, member added/removed, move (request, approval, rejection)
Approval	request, confirmation, rejection, cancellation, execution, break-glass
Settings and keys	settings changed (also exceptions and protection), key rotation, errors
Areas	area created/changed/deleted, manager added/removed, request, grant, denial, revocation, cancellation, refused access, keeper added/removed, protection enabled/disabled

PIN-only people	creation, invitation, sign-in, wrong PIN, new PIN, reset, revocation, deletion, blocking and unblocking, rules, unblock request, expiry and its warning, manager, working time

16. Settings

> **Who sees them:** CORE (master and deputies). Nobody else. (When there are keepers and active CORE deputies, a superuser who is not a CORE deputy does not see them.)
>
> **Who can change:** the master and a CORE deputy with the permission *Can edit settings*. Others see the page marked *Read-only view*.
>
> **Confirmation:** saving needs the PIN (and 2FA with *Require 2FA also for settings* on). With four-eyes approval and the category "settings" it creates a request.
>
> **When:** after the installation and when the company policy changes.

Menu **User PIN → Settings** (*PIN settings*). The page has the cards: General, Access & PIN policy, Rotation, Unlock, Protection, Four-eyes approval, Protected areas and at the bottom *Encryption key* (the active key, the fingerprint, the configured keys). **Mail** (SMTP, domains, notifications, recovery of a forgotten PIN) is on its own page *Mail*; **Deny everything not allowed** and the other protection options are on the page *Protected areas*; the rules of the PIN-only people are on the page *People with a PIN only*.

16.1 General

Option	Default	Meaning
Language	English	language of the plugin pages and e-mails: *English* or *Slovenčina*
Show first and last name	yes	the name next to the account (Users, CORE, Departments, e-mail recipients)

16.2 Access & PIN policy

Option	Default	Meaning
Who may use a PIN	only explicitly allowed	*Only explicitly allowed users* or *All users, except denied*. The per-user flag is on the Users page. Superusers always may
PIN length	6	number of digits, 4 to 12. Old PINs of another length stay valid until changed
Block weak PINs	yes	rejects repeated digits (111111), sequences (123456) and patterns (121212)
Additional blocked PINs	empty	an own list; one per line or comma separated

16.3 Rotation

Option	Default	Meaning
PIN max age (days)	180	1 to 3650; after it the PIN must be changed
Warn before expiry (days)	14	0 to 365; sends an e-mail once and shows a notice

16.4 Unlock

Option	Default	Meaning
Unlock duration (minutes)	15	1 to 1440
Extend unlock on activity	yes	every view of a protected page restarts the timer (it expires after inactivity)
Unlock scope	one unlock for everything	or a separate unlock per scope (plugin / area)
Require 2FA on every unlock	no	the PIN and a 2FA code

16.5 Protection

Option	Default	Meaning
Max failed attempts	5	1 to 50; PIN and 2FA are counted separately
Lockout duration (minutes)	30	1 to 10080 (7 days)
Require PIN + 2FA for administrative actions	yes	allow/deny, reset, force change, settings and delegates need a fresh confirmation; when off the PIN is enough
Require 2FA also for settings	no	off: Settings, Mail and everyday actions need only the PIN. Resets, suspension and roles always need PIN + 2FA
Administrative window (minutes)	5	1 to 60; how long a confirmation is valid
2FA asked again after (hours)	8	1 to 168; in the administration of protected areas
Administrator reset valid (hours)	24	1 to 336; how long the user has to confirm a reset

16.6 Four-eyes approval

Option	Default	Meaning
Four-eyes approval	no	switching on needs 2 active CORE deputies; it cannot be switched off while a department exists
… for roles	yes	see the Approval chapter
… for settings and mail settings	yes	including adding a domain
… for allowing / denying PIN use	no	
Approval request valid (minutes)	1440	5 to 10080; applies also to move requests
Allow break-glass for the master	no	a superuser may execute a request alone with a reason

Saving reports an error if you switch four-eyes on without two active CORE deputies (*At least two active CORE deputies with PIN and 2FA are needed*) or off with a department existing.

16.7 Protected areas

Option	Default	Meaning
Protect areas and objects	no	the areas defined on the Protected areas page take effect. Plugins with their own "PIN integration" are protected when their own switch is on, even if this option is off. **It cannot be switched off while areas with managers are active**

16.8 What is on other pages

Option	Where
Security notifications by e-mail; Self-service recovery; Recovery code validity	Mail
Deny everything not allowed; What is not allowed (A / B); Users may ask for access; Users may create their own API key	Protected areas (the Protection card)
Address of the PIN-only sign-in; wrong PINs; e-mails; working time	People with a PIN only

There is also a hidden option *Whom the protection applies to* (by default **only people with a PIN**: the PIN-only people and the users in a group or section managers; everybody else works by NetBox permissions and does not see the plugin). It cannot be changed in the interface.

17. Protected areas

Who: the page *Protected areas* is seen by a protection keeper, a section manager and (while there is no keeper) CORE / the master. Areas are created and targets added only by a **keeper** (while there is none, CORE / the master). Groups, fields and members are managed by a keeper or the manager of the section.

When: when a part of NetBox (an app or an object type, e.g. DCIM › Device, Tenancy › Tenant) is to be seen and changed only by those who have been given access.

Depends on: the protection switched on (the Protection chapter), at least one keeper, section managers with a PIN and 2FA, and for requests mail (the E-mails chapter).

17.1 The basic idea

An area protects an **object type** (`dcim.device`) or a whole **app** (`dcim`). In a protected area **nobody is a god**: the plugin does not take the role or the NetBox permissions of the user into account at all, the **group** into which a keeper or a section manager put the user decides, even for a superuser. The door (a menu entry, a tab) stays visible for everybody; whoever has no key is refused on entering and learns who manages the access (the page "This area is protected"). Sensitive apps (`users`, `auth`, `core` etc.) can be put into an area only by a keeper.

17.2 The Protected areas page

- **The Protection card**: the ON/OFF state and the switch (the Protection chapter).
- **Protection keepers**, **Exceptions**, **What the users see**: the Protection chapter.
- **What is set up**: the list of protected apps and object types. Each has the number of members, groups, managers (*no owner yet*) and the buttons *Open* and *Delete* (*Remove the protection of this object type? Its groups and accesses are deleted.*).

- **Add an app or an object type**: choose from the list and *Add*. Adding only prepares the structure: **nobody gets access** until you create a group and put users into it. Until an area has a manager only the keepers see it.
- At the top is the number of *access requests waiting for your decision*. If the protection of your own areas is off the page says so (*Settings → Protected areas*); areas of plugins with a "PIN integration" protect as soon as the plugin's integration is switched on.

17.3 The area detail

Part	What it is
Waiting for a decision	access requests (user, when asked, what asked for, why, what exactly). Buttons *Decide* and *Deny*
Groups	named sets of rights. *New group* → name → *Create the group*; open it and set the rights, tabs, fields and members
Users	who is in which group, *until* when (or *no time limit*) and the button *Remove*
Section managers	who decides about the area; see the chapter Section managers
Settings of the area	name, description, *Longest temporary access (hours)* (24 by default, at most 720), *Active*, *What is not allowed* (as the settings say / A door / B hidden): applies to the menu items and tabs of this area. The button *Delete the area*: the objects return under the NetBox permissions

A group is for one object type. A user is in **one group per type**: adding him to another moves him and leaving the last one means no access. An access can be temporary (*until*) or unlimited; a temporary one lasts from 1 hour up to *Longest temporary access* hours. A user may end their own access (*My access → End my access*); others' are removed by a keeper or a manager.

17.4 A group: rights

The group page (*Group*) has these parts:

Part	Options
Name of the group	text
Access	*Sees nothing* / *Read only* / *Read and write*. Whoever may write may read as well
Further rights	*May delete*; *May import (needs adding as well)*; *May export*. Without them the Import / Export buttons are not drawn and the pages answer "no access"
Rights of a plugin	if a plugin defines own rights (e.g. *config*) they are offered as further check boxes
Deny (*wins over any allowance*)	a right the group never has, even if it would get it otherwise
Tabs of the page	for each system tab of the detail (Interfaces, Config Context, Changelog …) *Hidden* / *Visible*. The main page is always allowed. Tabs of plugins follow the area of that plugin
Members	*Add users to the group*; the button *Remove*

A tab that lists **other objects** (a device's interfaces) needs the right on their type as well; with the protection on, that type must also be in some area.

17.5 A group: the records it sees

Option	Meaning
Not set (sees no record)	the starting point: the members see no row
All records	the members see all rows of this type
Only by conditions	they see only the rows that match the conditions

- **Allowed**: several values of one condition are alternatives (*agg* or *core*); several conditions must **all hold at once**. With no condition the group sees nothing.
- **Denied**: records that match any of these conditions are **never shown**, whatever else allows them.
- A condition = a *field* + an *operator* + a value. Operators: *is*, *is not*, *contains*, *does not contain*, *starts with*, *is one of*, *is not one of*. For a field with choices or a relation the values are chosen from a list.

17.6 Fields

For a group (or for everybody in the area) and an object type every field is:

Mode	Meaning
By rights (default)	whoever may add or change sees and writes every field; a read-only user reads
Hidden	the field is not shown to the user
Read only	visible, cannot be changed
Read and write	visible and changeable

- The **Fields** page (*Rules for* a group or user + an object type): unfold the group and set field by field, or all at once with *Set all shown fields* (applies to the fields left by the search), then *Save rules*.
- The option **Hidden from everybody in the area** hides the field from everybody in the area, managers included.
- **Custom fields linked to other objects** can be limited to the values matching a condition (*Values must include*, or the *Conditions* button: *contains*, *starts with*, *is one of*; several values as chips). For example the user picks only devices whose name contains `PRINTER`. The condition is also checked on saving. It does not apply to fixed fields.
- A hidden custom field **keeps its stored data** (saving an object never wipes it).
- **Helper fields of a form.** Some fields of a form are not fields of the model, for example *Owner group* next to *Owner* (it only narrows the choice of owners and stores nothing). They are not in the list of fields; they follow the field they belong to: hiding or locking *Owner* hides or locks *Owner group* too.

Where the rules apply	custom fields	model fields
object detail	hidden / value filter	**not covered** (NetBox draws the fixed fields by hand)
edit forms	removed / disabled; the choice is narrowed	removed / disabled
tables	column removed	column removed
REST API	omitted / filter / writes ignored	omitted / read only

17.7 Tabs of the detail

The page *Tabs of the detail page* decides which tabs of an object's detail (Interfaces, Config Context, Changelog …) the user may open. The main page is always allowed. A new access allows only the main page. A section manager opens every tab.

17.8 An access request

1. A user whose way is closed sees the page "This area is protected" (who manages the access, what was blocked, what you can do here: *Change the password*, *My PIN*, *My API keys*). They tick **what they need** (view / add / change / delete), write **why** and **what exactly** (section, group, object) and press *Request access*.
2. An e-mail with a link goes to the **section manager and the keepers**; if no area exists for the type yet, only to the keepers. The link opens the page *Access request* (who, what, why, when).
3. The decider chooses the **group** to put the user into (or *A new group* with a name), the **rights to grant** and **for how long**, and presses *Approve* or *Deny*. If no area exists for the type yet, the approval creates it (without managers: they must be appointed by hand) and adds the user.
4. The requester gets an e-mail about the decision (so do the keepers and the manager). A waiting request can be **cancelled** in *My access → My requests*.

- Nobody decides about their own request. Only one request can wait per user, area and type.
- Requests can be switched off (*Users may ask for access*): then only the administrators make them. PIN-only people cannot ask.
- A user without a PIN is sent to create one on entering (no 2FA needed) and then continues.

17.9 My access

Menu *My access*: **My access to protected areas** (area, profile, until, *End my access*), the sections you manage and **My requests** (area, when, status, *Cancel*). At the top is the number of requests waiting for your decision.

17.10 What is enforced

Where	How
NetBox pages (lists, details, add/edit/delete, bulk actions, plugin pages)	middleware refuses without the right (*view* for reading, *add / change / delete* by the kind of page); the PIN has to be unlocked
Rows	wrapped `RestrictedQuerySet.restrict`: in a protected area it returns the rows the group allows (none without one): lists, details, REST API, drop-downs
REST API and GraphQL	refused without the right (403 JSON)
Menu and tabs	`has_perm` is answered by the accesses from the PIN
Change log, journal, attachments	never show rows of objects a user may not view

Not covered: filters in lists (`?cf_name=…`), exports, the fixed fields of the detail and lists mixing several types. Anybody with root / database access on the server can bypass any plugin: the answer is the audit, not a technical lock.

17.11 An area of a plugin

A plugin can register its own area (`areas.register(...)`) with its own rights and profiles. It is protected when its "PIN integration" switch is on, even without the global *Protect areas and objects*. See the chapter Using it from another plugin.

18. Global protection (ON / OFF) and keepers

Who: only **protection keepers**. While there is none, the first one is appointed by CORE / the master.
When: when everything that no group allows is to be denied; it is switched off in an emergency or for maintenance.
Depends on: at least one keeper with a PIN and 2FA. Without a keeper the protection cannot be switched on.

The **Protection** card is at the top of the *Protected areas* page.

18.1 ON and OFF

State	What applies
ON	everything that no group allows is denied to **everybody except keepers and exceptions**. PIN-only people see only their groups. It reaches every path: lists, details, REST API, GraphQL, search, widgets, menus and buttons. A superuser who is not a keeper sees none of Users, Groups, Permissions, Jobs, Background tasks… The own profile, bookmarks, notifications and preferences stay open, but only the user's own
OFF	everybody works by the NetBox permissions alone. PIN-only people can only change their PIN and are told that the function is switched off by the administrator

The buttons *Switch the protection ON* / *Switch the protection OFF* are seen and pressed only by a keeper. The change is written to the audit log and an e-mail goes to the other keepers.

18.2 Protection keepers

- The card **Protection keepers**: the list, *Remove* and *Appoint another keeper*. While there is none, *Appoint the first keeper (switches the protection on)* stands there: **appointing the first keeper switches the protection on**.
- A keeper must have a **PIN and 2FA**, the account must be active and must not be a PIN-only person. After the appointment the e-mail *You are a protection keeper* arrives.
- A keeper is **exempt from the denial** (NetBox permissions apply to them), cannot turn off 2FA and **cannot leave alone**: another keeper has to take over.
- **The last keeper cannot be removed.** If the account of the last keeper is deleted, the protection goes OFF and cannot be switched on until a keeper exists.
- **A keeper can do everything**: areas, object types, section managers, exceptions, the ON/OFF switch and all PIN-only accesses. The limits for exceptions and managers do not apply to them.

18.3 What the users see

The card **What the users see** (saved with *Save*):

Option	Default	Meaning
What is not allowed	A – door	**A – door:** the user sees everything and the PIN blocks what is not allowed with a notice (and a request). **B – hidden:** a user who is in no group sees nothing (only the change of the password and PIN); in a group they see only what the group allows; no API in the browser. Every area may choose its own (Settings of the area)
Users may ask for access	yes	off: only the administrators make requests (there is no page or button)
Users may create their own API key	no	a key works with the NetBox permissions and the PIN does not judge it: a superuser with a key reads everything through a script. Allow it only for narrow permissions. A user then sees only their **own** keys

> There is also a hidden option *Whom the protection applies to*: by default **only people with a PIN** (PIN-only people and users in a group or section managers; everybody else works by NetBox permissions and does not see the plugin). It cannot be changed in the interface.

18.4 Emergency on the server

```
python manage.py userpin_keeper                # shows the keepers
python manage.py userpin_keeper --add <name>   # appoints a keeper
python manage.py userpin_default_deny --on     # switches the protection on (needs a keeper)
python manage.py userpin_default_deny --off    # switches the protection off
```

Every change by a command is written to the audit log and **all keepers and superusers learn of it by e-mail** (nothing happens quietly).

19. Exceptions

Who: only a keeper sets, changes and removes. A regular user (not a keeper, not a PIN-only person) can have an exception.
When: when one user is to work by NetBox permissions without the denial (e.g. a network administrator whom the protection limits).
Depends on: the protection switched on (while it is off nothing is taken away).

A user with an exception is not bound by the protection: they work with the permissions NetBox gives them. Their groups, accesses and roles stay as they are and apply again when the exception is taken away or ends. A superuser with an exception sees everything without the PIN. The **Exceptions** card is on the *Protected areas* page.

19.1 An exception on and off (the switch)

At every user there is a **slide switch**. The user stays in the list with all the settings (what they may use, the reason, the end) and you switch the exception on and off with one click, **without removing it**:

Switch	What applies
Exception on	the user is ruled **by NetBox** and of the account pages and the API may use only what the keeper ticked (*May use*)

Exception off	the user is ruled **by the PIN**: the protection applies to them like to a regular user (groups, denials). The list of allowances is not used then, but it stays stored and applies again when switched on

- Only a keeper can switch. The change takes effect at once (within 5 seconds at the latest), is written to the audit log and the other keepers get the e-mail *Protection exception switched on / off*.
- Every row in the list has a tag *Exception on* (green), *Exception off* (yellow) or *Ended* (grey) and under the switch a text saying who rules them now (*NetBox rules him, only what is allowed* / *The PIN rules him*).

19.2 The end of an exception (date and time)

- At every user there is the field **Until** with a date and time and a *Save* button: you can change the end at any time. **An empty field = no end.** A new end must be in the future.
- **When the end passes** the PIN rules the user again at once (within 5 seconds) and the system job (every 5 minutes) moves the switch to *Exception off* and writes it to the audit log; the other keepers get the e-mail *Protection exception ended*. The row stays in the list with the tag *Ended*.
- **You switch an ended exception on again** by first setting a new end (or clearing it) and then switching it on. Switching on with a past end is refused (*The exception has ended: set a new end (or none) before switching it on*).

19.3 Searching and a list for many users

- Above the list there is **Search** (name, account or reason) and the filter **State**: All / Exception on / Exception off / Ended. The *Clear* button resets it. After changing a switch or an end the search is kept.
- The list is **paged by 25**. At the top there is a summary (*N exceptions: X on, Y off*).
- When adding there is a **search field** above the choice of the user: you type and the list narrows; when one is left it is chosen by itself.

19.4 Adding an exception

The form: *User* (keepers and those who already have an exception are not offered), *Until (optional)* (the end; must be in the future), *Why (optional)* (up to 200 characters) and the button *May use* with the items (see below), then *Set the exception*. A new exception is **on**. The other keepers get the e-mail *Protection exception: name* and it is written to the audit log. If the user is in groups the page warns that they do not apply while the exception lasts. Adding the same user again is refused (changes are made in the list).

19.5 What a user with an exception may use

The exception concerns the protection, not the account pages and the API. Those the user may use only as the keeper allowed:

Item	Default	What happens when not allowed
Change the password	allowed	the Password tab and the link in the menu disappear, the address returns 403
Profile	not allowed	the tab and the address `/user/profile/`
Preferences	not allowed	`/user/preferences/`

Bookmarks	not allowed	`/user/bookmarks/`
Notifications	not allowed	`/user/notifications/` and the bell is hidden
Subscriptions	not allowed	`/user/subscriptions/`
API tokens (create and manage their own)	not allowed	`/user/api-tokens/`, `/users/tokens/`, `/api/users/tokens/`; the item *Admin → API Tokens* disappears too
API (REST, GraphQL) with a token	not allowed	a token of such a user is refused (an authentication error). The browser itself (UI) works through the session and is not affected

- Under the name of every user there is an arrow ▾ **May use** that unfolds the check boxes with a *Save* button, and small tags: green ✓ allowed, red ✗ not allowed.
- What is not allowed disappears from the menu and the tabs and the direct address returns 403 (*Not allowed for this account*, with the names of the keepers).
- NetBox has no link to the password in the user menu (only a tab of the profile). While the password is allowed, the plugin adds one to the menu.
- **A keeper can always do everything.** While the protection is off nothing is taken away. A superuser who is not a keeper is not limited either while there is no keeper. With the exception *off* the list of allowances is not used (the PIN rules the user).

Why it matters. NetBox gives every signed-in user the right to their own API tokens (`DEFAULT_PERMISSIONS`), so a user without any rights sees *Admin → API Tokens* in the menu. A token carries only the permissions of its owner, but the plugin does not judge requests with a token: a user with rights could read beyond their areas through a token. The list of allowances solves it without touching the NetBox configuration.

20. Section managers

Who appoints and removes: a keeper (while there is none, CORE / the master). A section manager does not appoint or remove managers.

When: when somebody other than a keeper is to decide about a part of an area: groups, fields, members, requests.

Depends on: the manager must have a PIN and 2FA and be a regular NetBox account (not a superuser, not a keeper, not a PIN-only person).

- A **section** = the object types of the area; **nothing ticked = the whole area**. At a manager there is the button *Save the section*; you appoint a new one with *Appoint the section managers* (the form *Section* + a choice of users).
- A manager has **full power over their section**: groups, fields, members and requests. The keepers stand above them. A department role is not needed.
- Requests for their types go to **them and the keepers**; every decision is e-mailed to the requester, the keepers and the manager.
- PIN-only accesses made by a keeper belong to the keepers; a manager sees only those **under them** and creates them only into the groups of their section (the keepers get an e-mail). A keeper can move an access under a manager.

- A manager of a whole area keeps their rights over the area (targets, profiles); creating an area, adding and removing targets and managers belong to the keepers. **The last manager of an area** can be removed only by a keeper (CORE / the master while there is no keeper cannot remove them, except a manager who has meanwhile become a superuser or a keeper).
- Appointing a manager automatically **allows them to use a PIN** (unless the PIN administrator explicitly denied it: then the appointment fails).
- When appointing, the page shows what the manager lacks (*Has a PIN*, *Has 2FA*, *Is not a superuser or a protection keeper*, *Is a NetBox account*).
- Emergency on the server: `python manage.py userpin_area_owners <area-key> --set name1 name2` (written to the audit log; it refuses people who could not be managers unless you give `--force`).

21. People with a PIN only (PIN-only)

Who creates and manages them: a protection keeper (all) and a section manager (only those under them and into the groups of their section). The menu *People with a PIN only* is seen by a keeper and a manager.

When: for people who have no NetBox account and are to see or change only a part (e.g. a technician on tenants).

Depends on: mail set up (the personal link comes by e-mail; without it you see it on the screen), a group in a protected area, the protection switched on (while it is off the person can only change the PIN), a running `netbox-rq` (summaries of changes).

Behind a person stands a technical user `PinUser0001`, `PinUser0002` … (a number is never reused or changed; the *alias* and description are free texts and nothing is renamed in NetBox). The technical account has **no usable password**: it can sign in only through the PIN page. Nobody sees it in user lists, drop-downs or the API. The NetBox change log shows the technical user.

21.1 A new access (the form)

Field	Required	Meaning
Name	yes	the person's name (up to 150 characters)
E-mail	yes	a valid address. It **must not** belong to another access or a NetBox account: if an account already has it, the page explains that the person should sign in with their own account (add it to the group as a regular user)
Group	keeper no; manager yes	a group of a protected area. Without a group (keeper only) the person can only change their PIN. A manager chooses only from the groups of their section
Access ends (optional)	no	a date and time in the future; afterwards the person cannot sign in. Empty = no end
Idle (minutes)	–	30 by default (5 to 240)
Session (hours)	–	8 by default (1 to 24)
Allowed IP addresses (optional)	no	addresses or networks (e.g. `10.0.0.0/24`), separated by a comma, semicolon or newline; an invalid entry is refused
Alias, Description (optional)	no	alias up to 100, description up to 150 characters

PIN language; NetBox language	no	default / Slovenčina / English; the language of the PIN pages and e-mails, and the language of the NetBox interface for this person
Working time	no	the general rules / own rules / no limit (see the Working time chapter)
Send to the person (every change of the access)	–	see below

21.2 The person's first sign-in

1. The person gets a **personal link** by e-mail (valid **7 days**; *Send the link again* issues a new one and the old one stops). The link opens the page *Hello name*.
2. They choose **their own PIN**: **8 to 64 characters** (not only digits), at least 4 different characters, **unique** among all accesses (*Choose another PIN* if somebody already has it). Nobody can read it, not even the administrator.
3. The button *Set the PIN and sign in*. If the invitation expired: *Ask the administrator for a new one.*

If mail is not set up, the link is **shown to the administrator on the screen** (once, *Give this personal link yourself*).

21.3 Regular sign-in: one shared address

- Everybody uses **one shared page** (`/pin/` by default) and enters **only the PIN**: the PIN tells who the person is. The address is chosen by a keeper on the page (*Address for signing in with a PIN*: part A and optionally B, e.g. `/tenants/` or `/aqer/x2/`; case-insensitive). The person saves it as a bookmark; it **does not change** until a keeper changes it (after a change they must save the new one).
- The personal link from the e-mail works too (the person is known there).
- If they forgot the PIN: *I forgot my PIN – send me a link to set a new one* (the address; the link is valid **1 hour, once**, at most one per minute; it does not work for a disabled or ended access).

21.4 What the person sees and cannot do

- The welcome page: *You are signed in as name*, the PIN validity (days and hours), the **working time** (*Your working time*), after the end *It is over: one more save, then you only read*, the inactivity after which the session ends, and the list of accesses (*group X, you may..., valid until ...*) with the button *Continue*. Without a group: *No access has been given to you yet. You can only change your PIN.* With the protection off: *The function is switched off by the administrator for now. You can only change your PIN.*
- The user menu: *My access*, *Change my PIN*, *Sign out*. They have no password, 2FA, API or token; they are never a keeper, a manager or an exception. The account pages and the pages of the plugin are closed to them. They cannot make access requests.
- The drop-downs of forms work for them (brief API lists); the rest of the API is closed. They save their own table columns.
- **Changing the PIN** (*Change my PIN*): the current PIN, the new PIN, the new PIN again. After the expiry or a forced change it is asked before continuing.

21.5 Wrong PINs

The rules are *Wrong PINs per step* (5 by default; 1 to 50) and *Waits after each step (seconds)* (`10,20,30` by default; 1 to 10 waits of 1 to 3600 s). They are set for everybody (*Rules for wrong PINs (defaults)*) and may also be own for one person; empty = the defaults.

Where	What happens
The personal link	the person is known: after every 5 wrong PINs a wait (10, 20, 30 s); after the last step the access is **blocked** (the person and the keepers get an e-mail). Only an administrator unblocks (*Unblock*), or the person sets a new PIN through an e-mail with a link
The shared page	a wrong PIN cannot be put down to a person: the **computer** (its address) waits by the same steps, the last wait repeats, **nobody is blocked**, and the keepers get an e-mail after every full round (15 wrong PINs) *Wrong PINs from ...*

A blocked person sees *Signing in is not possible* and the button *Ask the administrator to unblock my account* (at most once an hour; an e-mail to the administrator who disabled the access, or to the keepers).

21.6 Actions at a person

Action	What it does
Unblock	the old PIN works again, counting from zero
Send a reset e-mail	a link for a new PIN (valid 1 hour, once); the new PIN also unblocks
Force a PIN change	at the next sign-in a new PIN is asked
Send the link again	a new personal link (the old one stops)
Reset the PIN	the PIN is cleared and a **new personal link** is sent through which the person chooses a new PIN (without mail the link is shown on the screen)
Disable / Enable	disabling ends all sessions and blocks the sign-in (the person sees "disabled by X since ..."); enabling gives the access back
Delete	removes the access and the technical user

At every person there is the unfoldable **Activity and rules**: *Sign-ins and PIN events*, *Changes in NetBox*, the forms *Alias and description* (and *Language of the PIN pages and e-mails*, *Language of the NetBox interface*), *Manager* (the keepers; under a section manager they run it, a keeper always can; a move under a manager), *E-mails*, *Working time*, *Account valid until* (empty = no end; after the end the person cannot sign in; they get an e-mail 30 days and 1 day before) and *Rules for wrong PINs*.

21.7 The person's session

The session ends after *Idle* minutes (30 by default; 5 to 240) or after at most *Session* hours (8 by default; 1 to 24), at the end of the access validity, after disabling, or when a request arrives from an address outside the allowed IP list. It is set for every access separately. After signing out the person returns to the shared PIN page.

21.8 E-mails both ways

Direction	What	Setting
To the person	every action of an administrator: groups, disable / enable, PIN reset, unblock, validity, languages, rules, working time, manager, alias/description, deletion. The mail goes to their own address (whatever the allowed domains are) in their language	*Send to the person* (the whole page) and *Send to the person (every change of the access)* at an access. If it is off on the page, **that wins**

To the administrators	what the person added, changed or deleted in NetBox (with the changed fields), to the administrator who created the access and to the keepers	*Send to the administrators* (the whole page) and per access

The administrators get the changes either as a **summary** every N minutes (*Summary every*: 5, 15, 30, 60, 120, 240 minutes or once a day; 60 by default), or **at every change** (one e-mail per change): **never both at once**. The subject at every change: *PinUser0001 made a change*; the body: time, object and the changed fields (from → to). The e-mail is sent after the save through the `netbox-rq` queue, so saving stays fast; the summaries are processed by the system job every 5 minutes. A keeper sets it in the card *E-mails (for everybody)*.

> When a section manager works with an access, the keepers are told by e-mail (nothing happens quietly). An access made by a keeper belongs to the keepers until it is moved under a manager.

22. Working time

> **Who sets it:** a keeper (the general rules and the rules of the individual accesses).
> **When:** when PIN-only people are to work only in working hours.
> **Depends on:** *Limit the working time of the PIN-only accesses* switched on (off by default), the `holidays` library for the holidays, the time zone of the server (`TIME_ZONE` in NetBox).

22.1 The rules

The general rules are on the *People with a PIN only* page (the card **Working time (for everybody)**): *Limit the working time*, *Country (holidays)* (SK by default), *Days* (Mon–Fri by default), *From* (06:00) and *To* (16:00). At every access you can choose:

Option	Meaning
the general rules	the rule of the page applies (if the limit is on; otherwise no limit)
own rules	an own country, days and hours (e.g. a Hungarian technician with the Hungarian holidays)
no limit	the person has no limit at all

Countries: SK, CZ, HU, AT, PL, DE, UA, HR, SI, RO, BG, RS, GB, IE, FR, ES, IT, NL, BE, CH, SE, NO, DK, FI, PT, GR, LT, LV, EE, US, CA. The public holidays come from the `holidays` library (works offline, needs `pip install holidays`; without it the page reports *The library of public holidays is not installed* and only days and hours are checked). The holidays follow the law the library implements: for example in Slovakia in 2026 the days off are 1 Jan, 6 Jan, Good Friday, Easter Monday, 1 May, 5 Jul, 29 Aug, 1 Nov and 24, 25, 26 Dec, while 8 May, 1 Sep, 15 Sep and 17 Nov are working days. The plugin has no vacations or custom company days.

22.2 What happens outside the time

- The person **can sign in and read**, but **cannot save or change anything**: no saving, adding, editing, deleting or writing through the API. Forbidden requests are redirected with a message (*Outside your working time (…) you can only read*), the API answers 403.
- The working time is **always visible**: on the welcome page and in the top bar, in the person's language. On a holiday *Today is a public holiday (name)* is written before it.
- When the time ends **while they work**: **30 minutes and one save**, then only reading.

- The state of a day: in time / just ended (30 minutes) / before the start / after the end / a day off or a holiday.

23. Server commands

Who: the server administrator (root).
When: at installation, key replacement, in an emergency (a lost phone, a locked protection, non-working administrators).
Depends on: the NetBox virtual environment activated. Emergency commands are written to the audit log.

They are run like this (replace `<command>`):

```
sudo bash -c 'source /opt/netbox/venv/bin/activate && cd /opt/netbox/netbox && python manage.py <command>'
```

Command	What it does
`userpin_generate_key [--key-id k1]`	generates a random encryption key (prints it, stores nothing)
`userpin_rotate_key`	re-encrypts everything under the active key (PIN hashes, 2FA, backup codes, SMTP password); in one transaction, on an error nothing changes
`userpin_check`	verifies that everything stored under the keys can be decrypted and shows how many values and PIN-only people wait for an old key
`userpin_keeper [--add name]`	shows the keepers or appoints one (audit, e-mail to all keepers and superusers)
`userpin_default_deny [--on | --off]`	shows or switches the global protection (audit, e-mail; switching on needs a keeper)
`userpin_area_owners <area> [--set names…] [--force]`	shows or replaces the managers of an area (audit); without `--force` it refuses people who could not be managers
`userpin_reset_2fa <user>`	removes the 2FA in an emergency (e.g. the master without the phone and backup codes; audit)
`userpin_access_test <user> [--url …] [--all] [--writes]`	shows what a user gets from the pages, the REST API (browser) and the lists (a test of the protection). `--url` adds one more address, `--all` tries every NetBox page, `--writes` also an add, a change and a delete

24. Check: pin_check.sh

Who: the administrator; the script can also be run from a computer other than the server.
When: after the installation, after updating the plugin and NetBox.
Depends on: `curl`; with `--server` root on the NetBox server.

The script `tools/pin_check.sh` (in the repository) checks everything that can be checked from outside and with `--server` also on the server. It never stores or prints a PIN or a token (the PIN is asked silently). The result is the number of **OK** / **FAIL** / **WARN**.

```
bash pin_check.sh http://172.30.1.25                                  # anonymous checks only
bash pin_check.sh http://172.30.1.25 --pin /pin/                      # + signs in a PIN-only perso
n
NB_TOKEN=xxxx bash pin_check.sh http://172.30.1.25                    # + checks with an API token
sudo bash pin_check.sh http://127.0.0.1 --pin /pin/ --server          # + migrations, services, key
s, keepers
```

Options: `--pin <path>` (the address of the PIN-only page, `/pin/` by default), `--wrong` (tries **one** wrong PIN; it counts as a wrong attempt), `--write` (tries to add a tenant as the PIN-only person and deletes it), `--server` (needs root). Variables: `NB_TOKEN`, `NB_VENV`, `NB_DIR`, `LIST`.

The checks have sections: 0 reachable; 1 nobody signed in; 2 with an API token; 3 a PIN-only person; 4 what the person must never reach; 5 a page the person may open; 6 adding as the PIN-only person; 7 sign out; 8 on the server (migrations, services, keys, keepers). Known harmless warnings: `/static/docs/` is served by nginx (closed with `location /static/docs/ { return 404; }`) and `LOGIN_REQUIRED` is not `True` (set it in `configuration.py`; NetBox 5.0 removes it).

25. Security model

```
PIN ──Argon2id + salt──► hash ──AES-256-GCM (key from the configuration, bound to the user id)──► d
atabase
```

- A database dump alone contains nothing usable, not even something to brute-force offline.
- The ciphertext is bound to its user: a hash copied into another user's row does not decrypt.
- Wrong attempts are counted under a row lock, so parallel guessing cannot bypass the lockout.
- A PIN change or reset invalidates every unlock. TOTP secrets are encrypted the same way; a TOTP code is accepted only once.
- Backup codes (100 bits) and e-mailed codes are stored only as digests, single use, short-lived.
- Recovery never bypasses 2FA. The PIN is only an authentication factor ("it is really you at this computer"); it is not used as an encryption key for other data.
- **Requests with an API token are not judged by the plugin's PIN** (a token carries NetBox permissions): that is why own tokens are off by default and are allowed one by one for exceptions.
- Access to the server is ruled by the firewall; the plugin does not enforce a separate port or hostname for the PIN-only sign-in, if needed it is solved in nginx and the firewall.
- Anybody with root or database access on the server can bypass any plugin: the answer is the audit.

26. Using it from another plugin

Who: the developer of a NetBox plugin.

When: when another plugin is to protect sensitive pages with a PIN.

Depends on: `netbox-user-pin` as a dependency of the plugin; the API version (`service.API_VERSION` = 2, `areas.API_VERSION` = 1; the API only grows).

```
from netbox_user_pin.mixins import PinRequiredMixin, StepUpRequiredMixin, pin_required, step_up_req
uired

class ProjectView(PinRequiredMixin, generic.ObjectView):
    pin_scope = 'projects'          # used when the scope mode is "separate per scope"

class ProjectDeleteView(StepUpRequiredMixin, generic.ObjectDeleteView):
    ...                             # a sensitive action: a fresh PIN + 2FA

@pin_required(scope='projects')
def download(request, pk): ...
```

The user is redirected to *set PIN*, *change PIN* (expired) or *enter PIN* and then back. HTMX requests get an `HX-Redirect` header.

The service API `netbox_user_pin.service`: `has_pin(user)`, `is_unlocked(request, scope)`, `unlock(request, pin, scope)`, `verify_pin(user, pin)`, `lock(request)`, `has_step_up(request)`, `has_2fa(user)`, `is_allowed(user)`. Signals (`netbox_user_pin.signals`, `sender=UserPin`): `pin_set`, `pin_changed`, `pin_reset`, `pin_unlocked`, `pin_failed`, `pin_locked_out`, `pin_locked`.

26.1 An area of a plugin

```
from netbox_user_pin import areas

# in ready(): the area is active while the plugin's own "PIN integration" switch is on
areas.register('netbox_libreoxi', 'LibreOXI', targets=['netbox_libreoxi'],
               rights={'config': 'View device configurations'},
               profiles=[('Logs only', ['view']), ('Configurations', ['view', 'config'])],
               enabled=lambda: LibreOXISettings.objects.first().pin_integration)

# in a view, for the finer rights
if (response := areas.require(request, 'netbox_libreoxi', 'config')) is not None:
    return response            # the "no access - request it" page
areas.can(request.user, 'netbox_libreoxi', 'config')   # True / False
```

27. Troubleshooting

Message / symptom	Cause and solution
`ModuleNotFoundError: No module named 'argon2'`	With `--no-deps` the dependencies were not installed: `pip install "argon2-cffi>=23.1" "cryptography>=42" "segno>=1.5" "holidays>=0.50"`.
`CommandError: No installed app with label 'netbox_user_pin'`	The plugin is not in the `PLUGINS` list in `configuration.py`. Add `'netbox_user_pin'` and repeat `migrate`.
NetBox does not start after a configuration change	A key error at start (see the Installation chapter), or an error in `configuration.py` (a comma, an apostrophe). See `journalctl -u netbox -n 80`.
pip: *holidays requires python-dateutil<3 but you have 3.0.0*	A harmless warning: `holidays` works with dateutil 3 (verify by listing the holidays). Do not install `python-dateutil<3`, you could break NetBox.
`userpin_check` reports an error	A wrong or missing key. Do not change `k1` until the plugin is confirmed to work.
E-mails do not arrive	See the chapter E-mails: when they do not arrive.

A role cannot be added: "has no e-mail in a verified domain"	The invitee has no address in a verified domain or mail is not set up. Verify the domain on the Mail page.
A user without rights sees Admin → API Tokens	NetBox standard (`DEFAULT_PERMISSIONS`). For an exception the plugin hides it unless "API tokens" is allowed.
403 on an account page for an exception	That part is not allowed: the keeper ticks it with the exception.
A group gives a user nothing	He has an exception: the group does not apply while the exception lasts.
A PIN-only person can only read	It is outside the working time (or a holiday), or the protection is off.
A PIN-only person sees nothing	No group, or the protection is off.
The summaries of changes of the people do not arrive	`netbox-rq` is not running or *Send to the administrators* is off.
The upgrade put a plugin back to an older version	The pinned tag/commit is missing in `local_requirements.txt`, or the line has no `@…` (then `main` is taken).

28. Changes

The version in the code is **2.0.0** (the GitHub release) and **2.0.1** (branch `feature/exception-allowlist`). The original base was 0.38.4 (commit 7bed7de); the interim numbers 2.0.1 to 2.2.2 in the commit history were working labels, everything is part of the 2.0.0 line and is split by topic below.

28.1 2.0.1: permissions of exceptions

- An exception is no longer "all or nothing": the keeper ticks what the user may use (the password allowed by default; profile, preferences, bookmarks, notifications, subscriptions, API tokens and the API with a token to be allowed).
- What is not allowed is hidden from the menu and the tabs and refused by the address (403); a token without the allowed API is refused. A keeper is never limited.
- A "Change the password" link in the user menu; the bell is hidden when notifications are not allowed.
- An arrow ▾ under the name of the user with check boxes and green/red tags; an existing user is not offered again and adding one again resets nothing.
- New field `PolicyException.allowed` (migration `0042`); Slovak translation.
- A **switch of the exception** (*Exception on / off*) at every user: an off exception lets the PIN rule the user without removing the row; an on one lets NetBox rule them.
- **Editing the end of an exception** (date and time, empty = no end); when it passes the switch moves to *off* by itself (system job, audit, e-mail to the keepers); an ended exception must be extended before switching on.
- **Searching and paging** of the list of exceptions (name, account, reason; state filter; 25 per page) and a search field when adding a user.
- New field `PolicyException.active` (migration `0043`).
- The field *Owner group* (a helper field of a form) is hidden and locked together with the field *Owner*.

28.2 Keys and security

- Key rotation covers 2FA, backup codes, the SMTP password and the index of the PIN-only people too; `userpin_check` verifies all encrypted values.

- Removing the PIN of a user without 2FA; `userpin_area_owners` checks eligibility.
- The master manages PINs while there is no CORE deputy, even with existing keepers.

28.3 The new model of protection

- The PIN rules only the PIN-only people and the users in a group or section managers; everybody else works by NetBox permissions (menu and API keys included).
- PIN-only people are no NetBox users (hidden from lists), have no requests and without a group can only change the PIN.
- Protection ON rules everybody but keepers and exceptions; OFF leaves NetBox alone and pauses the PIN-only people. The ON/OFF switch is in the interface (keepers only) and shows the real state. Exception vs. group: the group waits under the exception.
- Protection keepers: the last-keeper rule, emergency commands (`userpin_keeper`, `userpin_default_deny`).

28.4 Section managers and requests

- A section manager with full power over his section (the keepers above); PIN-only people under a manager; requests to the manager and the keepers with e-mails about the decisions.
- Appointed only by a keeper; relaxed eligibility (PIN + 2FA, no department role).

28.5 PIN-only people

- E-mails both ways: the person about the changes of an administrator; the administrators a summary every N minutes or at every change (not both), subject *PinUserNNNN made a change*, body: time, object, changed fields (from → to); sent after the save through `netbox-rq`.
- The drop-downs of forms work; saving own table columns.
- Conditions on relation fields: contains / starts with / is one of, several values, the Conditions button.

28.6 Working time

- General and own rules, a country with its public holidays (the `holidays` library), working days and hours; the working time always shown.
- Read-only outside the time; 30 minutes and one save after the end; the country in the language of the person, days as Mon–Fri.

28.7 Tools

- `tools/pin_check.sh`: a check from outside and on the server; the dependency `holidays` in `pyproject.toml`; Slovak translations; a prepared `local_requirements.txt` for upgrading NetBox within 4.7.x.

28.8 What the plugin does not do yet

- Vacations and custom company days in the working time.
- The API for regular users in a group is still governed by the group rights (not fully closed).
- A separate port/hostname for the PIN-only sign-in (solved in nginx and the firewall).